

CSCE 451/851 Programming Assignment 1

Writing a Unix Shell with Parsing, Forking, and Logical Operators

Fall 2026

IMPORTANT PRELIMINARIES

- DO NOT PUT THE `fork()` STATEMENT INSIDE AN INFINITE LOOP!!
- This means no `while(1)`, or equivalent, ANYWHERE in your code.
- Use a bounded for loop instead, for example: `for (int should_run = 0; should_run < 25; should_run++)`
- You may safely assume that files are not infinite, and therefore you can read lines of a file until the file ends. For example: `while(getLine(file, inputString) != false)`
- If we find an infinite loop in the submitted code, you'll get a 0.
- The program should exit on the `exit` command.
- Finally, you may not use the `system()` function.

1 Overview of Project (PA1 and PA2)

This project consists of writing a C (or C++) program to serve as a shell interface that accepts user commands and executes each command in a separate process. A shell interface gives the user a prompt, after which the next command is entered. The example below illustrates the prompt `osh>` and the user's next command: `cat prog.c`. (This command displays the file `prog.c` on the terminal using the Unix `cat` command.)

```
osh> cat prog.c
```

The above is an example of a simple command (i.e., it does not contain any operators). We will extend this program to execute more complex commands which contain one or more simple commands connected together by an operator such as:

```
osh> cat < code.c
osh> cat < code.c > out
osh> ps | cat > out
```

Your shell will support the following operators (much like the standard `csh` or `bash` shells in Unix):

- `>` – Redirect `stdout` to a file
- `<` – Redirect `stdin` from a file
- `»` – Append `stdout` to a file
- `&&` – Execute next command only on success
- `||` – Execute next command only on failure
- `;` – Execute next command regardless of success or failure
- `|` – Pipe `stdout` of one command into `stdin` of next

The functionality defined here for the above operators is basic, and may slightly deviate from the functionality defined by the standard shells available on Unix. But it is not the purpose of this project to write a fully functional Unix shell. Rather, it is to introduce and familiarize ourselves with various concepts useful in understanding operating systems and systems programming, such as file I/O, process control, and inter-process communication. As such, we will only code parts of the shell that help introduce these topics.

PA1

This project is broken up into two programming assignments. In PA1 you will add support for:

- Input parsing
- `fork` and `exec` to create new processes
- All logical operators listed above except `|` (pipes)

PA2

In PA2, you will build upon PA1 and add support for:

- Interprocess communication via pipes

2 Submission

Use Canvas to hand in your assignment. Submit a single zip file, `<UNL_username>_<pa#>.zip` (e.g., `jdoe2_pa1.zip`) containing only:

- source files
- makefile with (at least) targets:
 - **all**: this must compile your `osh` binary and must be the first target in your makefile.

Executing `make` in your project directory should produce an executable named `osh`. Remember to verify that your code compiles and runs on the SoC servers.

3 PA1: Evaluation and Points Distribution

The project zip contains 5 test scripts, alongside the corresponding expected output files. The table describes the test case, and the points awarded for each test case.

Table 1: Evaluation Rubric

File	Answer Script	Test Case Description	Points
1.singleCommand.txt	ea1.txt	Simple command with arguments (no operators)	18
2.simpleRedir.txt	ea2.txt	Single command with single redirector (input or output <code><</code> , <code>></code> , <code>»</code>)	18
3.moreRedir.txt	ea3.txt	Single command with multiple redirectors (input and output <code><</code> , <code>></code> , <code>»</code>)	18
4.logicalConditional.txt	ea4.txt	Two commands connected by a single logical operator (<code>&&</code> , <code> </code> , <code>;</code>)	18
5.moreLogical.txt	ea5.txt	More logical operators	18
makefile		The program compiles successfully on command <code>make</code>	10
Total			100

The test scripts can be used to test your program and are the test scripts we use for grading. A fully functional executable, `osh`, is included in the distribution zip file for this project and is executable on the SoC servers. You can use it to see what your outputs should look like.

3.1 Critical “Gotchas” for Grading

- Your code **MUST** compile and run on the SoC server. If it does not, you will receive a 0.
- Ensure that the output binary produced by the makefile is named `osh`.

- Points will be awarded only if your output matches the expected output **exactly**. Please conform to the error strings as specified in the expected output for malformed commands.
- You **MUST** implement the `-t` option (written as “`-t`” here). This is described below. If you do not, you will receive a 0.
- Although you need to implement the `-t` option, you must also retain full interactive shell capability. We will test that it functions as an interactive shell.
- The parser has debug information and may introduce extra blanks in the output, so you should adjust accordingly.
 - Your shell needs to work interactively as a shell without extraneous debug information. That is, it should only print command outputs and the prompt `osh>`.
 - Your output must match **exactly**, with no extra blank lines or spaces.
- Some commands behave differently when outputting to an interactive shell versus a file. For example, the `ls` command documentation states:

“If standard output is a terminal, the output is in columns (sorted vertically) and control characters are output as question marks; otherwise, the output is listed one per line and control characters are output as-is.”

Running the commands from the test scripts interactively may therefore produce outputs different from what appears in the `ea*.txt` files. However, your output should match exactly when run with the `-t` option.

3.2 Grading Procedure

On our end, after running `make` to compile your program, we will run:

```
./osh -t < testscripts/testscript.txt >& tmp ; diff tmp testscripts/ea.txt ;
```

for each test script in the `testscripts` directory. Note that the `>&` operator redirects `stderr` to `stdout` in `csh`. This redirection looks different if you’re using a `bash` shell, but for grading we will use `csh`. If there are no differences, you get all the points; otherwise, we deduct points according to the table above. `exec csh` (or `exec tcsh`, `tcsh` is an enhanced `csh`) and get a `csh` shell

We will dock points for the following:

- Having an infinite loop in your code.
- If your program does not run as an interactive shell.
- If, when running as an interactive shell, you print extraneous output (anything beyond `osh>` and the command output).

4 PA1: Detailed Discussion and Description

This assignment is based on a common programming assignment in some operating systems books — building a shell. To help make the project more manageable, we have broken PA1 into four phases.

4.1 Phase 1 — Parsing the Input String

The first step is to parse the input line and figure out what it means. Mainly, we need to:

- identify the name of the program,
- supply the correct arguments to the program,
- check if input or output needs to be redirected,
- check if the output/input is redirected from/to a file or another program,

- check if there are any logical operators that control the execution of programs,
- repeat the same step for each command in the input.

In order to achieve this, we must:

- correctly parse the input line by separating each token,
- identify separate commands and arguments for each command from the tokens,
- build a data structure to represent the parsed data,
- save additional information (about how it interacts with the next/previous command, if at all). This could include saving file handles, logical operators, etc.

With this data structure, we should be able to traverse the list with ease, retrieve arguments, or skip to the next commands as needed.

We must also be able to identify malformed commands. Since we are able to traverse the command structure, we should analyze which command/operator combinations make sense and which do not. Mainly, we need to identify if

- we have a null command (e.g., `osh > cat || file`)
- if there is no file after a redirector symbol (e.g., `osh > echo >`)

You are welcome and encouraged to write your own parser. However, since the goal of our course is not tokenizing, we have supplied you with a parser that does all the above for you — you just need to use it in your code.

4.1.1 Using the Provided Parser

With the handout we have included a parser written in C++ and created by a former GTA, Yutaka Tsutano, and modified by Justin Bradley. Read the `README` to learn more about how to use it. It is also available on a GitHub repository.¹ It does all the hard work of parsing, including detecting malformed commands.

4.2 Phase 2 — Forking a Child Process and Executing a Command

After building the data structure, start by executing simple commands (and ignore the logical operators for the time being). Then you can extend the program to handle file redirection. This would involve:

- extracting a simple command from the data structure (executable name and arguments),
- creating a new process for the executable,
- supplying correct arguments to the newly created process,
- waiting for the process to complete and collecting its exit code,
- outputting the result to the console (for now; later we will modify this to handle a redirection operator).

We will use the following functions:

- `fork()`
- `exec()`
- `wait()`

Please go through the man page for each of these. They are fundamental to process management in an operating system.

¹<https://github.com/jmb275/osh-parser>

4.2.1 Step 1

Use the `fork()` and `exec()` commands with their respective arguments, ignoring file redirects and pipes for now (treat pipes as “;”). You need to understand how `fork()` and `exec()` work, so be sure to study them and understand the basics. You should first `fork()`, then `exec()`. Otherwise, if you `exec()` in the parent process, the shell you are using will be replaced with the executed command and cease to exist.

Shown below is pseudocode to accomplish this. Note the four rules when using `fork()`.

1. Never place `fork()` in a `while(1)` loop.
2. Trap and exit if the `fork()` fails.
3. Always have an `exit()` in the child.
4. Place a `wait()` in the parent (unless we have multiple `exec`'s, i.e., pipes).

You have a choice of six `exec` functions: `execl()`, `execvp()`, `execle()`, `execv()`, `execvp()`, `execve()`. They are all slightly different, so read up on them and pick the one that makes your life easiest.

Pseudocode Example

```
/* Rule 1: Don't forget this should not go inside a while(1)
   loop anywhere in your program */

pid_t cpid = fork();
if (cpid < 0) { /* Rule 2: Exit if fork failed */
    fprintf(stderr, "Fork Failed\n");
    exit(1);
}
else if (cpid == 0) {
    exec(...);
    fprintf(stderr, "Exec Failed\n");
    exit(1); /* Rule 3: Always exit in child */
}
else {
    int status;
    wait(&status); /* Rule 4: Wait for child unless
                      we need to launch another exec */
    printf("Child complete\n");
}
```

4.2.2 Step 2

Next, modify your program to handle redirection operators. A program writes its output to `stdout`, which points to the console by default. In order to write to a file, you need to overwrite `stdout` to point to a file handle. To do this, modify your program as follows:

- when you read the command, check for the input/output redirector (e.g., `<` or `>`),
- if so, get the file name,
- open the file with the appropriate option (new file vs. append),
- after the new process is created, and before doing `exec()`, overwrite `stdin/stdout`.

In order to overwrite `stdin/stdout`, use the `dup2()` system call. Here is a good description of `dup2()` and its use in a shell-like scenario. Finally, it may be helpful at this point to add a loop to execute each command in the input line separately (ignoring the logical operators and pipes for now).

4.3 Phase 3 — Logical Operators

Handling logical operators is fairly straightforward. The execution of the command is determined by the exit status of the previous command. In the previous phase, we implemented the logic to collect the exit status, which we can now use to determine whether or not the current command should be executed. Here are the steps:

- check if the current and previous command are separated by a logical operator,
- if so, get the exit status of the previous command,
- based on the exit status and logical operator, determine whether you need to execute the current command or skip it,
- if you need to skip it, set the exit status of the current command to be the same as the previous command.

The above logic is sufficient to handle chains of logical operators.

4.4 Phase 4

Finally, in order to evaluate your program, the program should implement a switch `-t`. When used, the program does not print the `osh>` prompt to the console. This helps us grade more easily by not having to see the `osh>` prompt regularly. If this is not implemented you will likely not get any points as the output will not match the test scripts.

The exact command we will run (for example, for test 1) will be:

```
./osh -t < 1.singleCommand.txt >& tmp ; diff tmp testscripts/ea1.txt ;
```